

THIS DOCUMENT IS THE ONLINE-ONLY APPENDIX TO:

An Architectural Style for Self-Adaptive Multi-Agent Systems

DANNY WEYNS

KU Leuven, Belgium and Linnaeus University, Sweden

FLAVIO OQUENDO

IRISA - Université de Bretagne-Sud, France

ACM Transactions Journal Format, Vol. 2, No. 3, 09 2001, Pages .

In this appendix we give a formal definition of the patterns of the architecture style in π -ADL. The pattern specifications are ordered according to their occurrence in the paper. Each pattern starts with a primary presentation followed by the pattern specification.

A. VIRTUAL ENVIRONMENT

```
component VirtualEnvironment is abstraction ()
{
    type Focus is view[focusName : String, focusParams : Any].
    type Sense is view[agentId : String, focus : Focus].
    type Representation is Any.
    type Action is view[agentId : String, actionName : String,
        actionParams : Any].
    type Operation is Any.
    type Message is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : Any].
    type Observation is Any.
    type Observed is Any.
    type SynchronizationUpdate is Any.
    type Transmission is Any.
    type StateItem is view[name : String, val : Any].
    type StateItems is set[StateItem].

    port perceive is { connection sense is in(Sense).
        connection sensed is out(Representation). }
    port act is { connection action is in(Action) }.
    port communicate is { connection sendMsg is in(Message).
        connection receiveMsg is out(Message) }.
    port operate is { connection operation is out(Operation) }.
    port synchronize is { connection syncOut is out(SynchronizationUpdate).
        connection syncIn is in(SynchronizationUpdate) }.
    port observe is { connection observation is out(Observation).
        connection observed is in(Observed) }.
```

Permission to make digital/hard copy of all or part of this material without fee for personal or classroom use provided that the copies are not made or distributed for profit or commercial advantage, the ACM copyright/server notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists requires prior specific permission and/or a fee.

© 2012 ACM 0000-0000/2001/0000-0001 \$5.00

```

port transmit is { connection transmission is out(Transmission).
                  connection deliver is in(Transmission) }.

//composition
behavior is { compose {
    actionService is ActionService().
    and perceptionService is PerceptionService().
    and communicationService is CommunicationService().
    and state is State().
    and synchronization is Synchronization().
} } where {
    sense relays perceptionService::sense
    and perceptionService::readWriteState unifies state::readWriteState
    and perceptionService::sensed relays sensed
    and perceptionService::observation relays observation
    and observed relays perceptionService::observed
    and action relays actionService::action
    and actionService::readWriteState unifies state::readWriteState
    and actionService::operate relays operate
    and sendMsg relays communicationService::sendMsg
    and communicationService::readWriteState unifies state::readWriteState
    and communicationService::receiveMsg relays receiveMsg
    and communicationService::transmission relays transmission
    and deliver relays communicationService::deliver
    and synchronization::readWriteState unifies state::readWriteState
    and synchronization::syncOut relays syncOut
    and syncIn relays synchronization::syncIn
}
}

```

A.1 State

```

component State is abstraction()
{
    type StateItem is view[name : String, val : Any].
    type StateItems is set[StateItem].
    type StateRepository is location[set[StateItems]].

    port readWriteState is {
        connection stateTemplate is in(StateItems).
        connection readState is out(StateItems).
        connection writeState is in(StateItems) }.
        assuming { protocol is { (via stateTemplate receive StateItems.
                                true*.via readState send StateItems.)*
                                or (via writeState receive StateItems.)* } }.

    stateRepository : StateRepository.

    behavior is {
        write is function( state : StateItems,
                          repository : StateRepository :
                              StateRepository { unobservable. }).
        read is function( template : StateItems,
                          repository : StateRepository :
                              StateItems { unobservable. }).
        choose

```

```

{
  //read state
  {via stateTemplate receive state_template : StateItems.
   via readState send read(state_template, stateRepository).
   behavior().}
  or
  //write state
  {via writeState receive state_items : StateItems.
   stateRepository := write(state_items, stateRepository).
   behavior().}
}
}

```

A.2 Perception Service

```

component PerceptionService is abstraction ()
{
  type Focus is view[focusName : String, focusParams : Any].
  type Sense is view[agentId : String, focus : Focus].
  type Representation is Any.
  type Observation is Any.
  type Observed is Any.
  type StateItem is view[name : String, val : Any].
  type StateItems is set[StateItem].
  type PerceptionTypes is location[set[StateItems]].

  port perceive is { connection sense is in(Sense).
                     connection sensed is out(Representation). }
  port observe is { connection observation is out(Observation).
                   connection observed is in(Observed) }.
  port readWriteState is {
    connection stateTemplate is out(StateItems).
    connection readState is in(StateItems).
    connection writeState is out(StateItems) }.

  perceptionTypes : PerceptionTypes.

  behavior is {
    analyzePerception is function ( sense : Sense ) :
      StateItems { unobservable. }.
    virtualPerception is function ( items : StateItems,
      types : PerceptionTypes:
      Boolean { unobservable. }.
    generateRepresentation is function( sense : Sense,
      state : StateItems) :
      Representation { unobservable. }.
    generateObservation is function( items : StateItems ) :
      Observation { unobservable. }.
    analyzeObserved is function( observed : Observed ) :
      StateItems { unobservable. }.

    sense_items : StateItems.

    via sense receive sense_request : Sense.
    sense_items := analyzePerception( sense_request ).

```

```

    if virtualPerception( sense_items, perceptionTypes )
    then {
        //perceive local state repository
        unobservable.
        via stateTemplate send sense_items.
        via readState receive perceived_state.
        via sensed send generateRepresentation(
            sense_request, perceived_state ).
        behavior().
    }
    else {
        //perceive external environment
        unobservable.
        via observation send
            generateObservation( sense_items ).
        unobservable.
        via observed receive external_observed.
        observation_state :=
            analyzeObserved( external_observed ).
        via sensed send
            generateRepresentation( sense_request,
                observation_state ).
        behavior().
    }
}

```

A.3 Action Service

```

component ActionService is abstraction ()
{
    type Action is view[agentId : String, actionName : String,
        actionParams : Any].
    type Operation is Any.
    type StateItem is view[name : String, val : Any].
    type StateItems is set[StateItem].
    type ActionTypes is location[set[StateItems]].

    port act is { connection action is in(Action) }.
    port operate is { connection operation is out(Operation) }.
    port readWriteState is {
        connection stateTemplate is out(StateItems).
        connection readState is in(StateItems).
        connection writeState is out(StateItems) }.

    actionTypes : ActionTypes.

    behavior is {
        analyzeAction is function ( act : Action ) :
            StateItems { unobservable. }.
        virtualAction is function( state : StateItems,
            types : ActionTypes ) :
            Boolean { unobservable. }.
        updateState is function( state: StateItems ) :
            StateItems { unobservable. }.
        generateOperation is function( state : StateItems ) :

```

```

    Operation { unobservable. }.

    action_items : StateItems.

    via act receive action_request : Action.
    action_items := analyzeAction( action ).
    if virtualAction( action_items, actionTypes )
    then {
        //apply action to local state
        unobservable.
        via writeState send updateState( action_items ).
        behavior().
    }
    else {
        //apply external action
        unobservable.
        via operate send generateOperation( action_items ).
        behavior().
    }
}
}

```

A.4 Communication Service

```

component CommunicationService is abstraction ()
{
    type Message is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : Any].
    type Transmission is Any.
    type StateItem is view[name : String, val : Any].
    type StateItems is set[StateItem].
    type Addresses is location[set[StateItems]].

    port communicate is { connection sendMsg is in(Message).
        connection receiveMsg is out(Message) }.
    port transmit is { connection transmission is out(Transmission).
        connection deliver is in(Transmission) }.
    port readWriteState is {
        connection stateTemplate is out(StateItems).
        connection readState is in(StateItems).
        connection writeState is out(StateItems) }.

    addresses : Addresses.

    behavior is {
        analyzeMessage is function ( msg : Message ) :
            ItemStates { unobservable. }.
        localCommunication is function( msg_items: StateItems,
            addrs : Addresses ) :
            Boolean { unobservable. }.
        generateTransmission is function( msg_items : StateItems,
            addrs : Addresses ) :
            Transmission { unobservable. }.
        analyzeTransmission is function( trans : Transmission,
            addrs : Addresses ) :
            StateItems { unobservable. }.
    }
}

```

```

generateMessage is function( trans_items : StateItems,
  addrs : Addresses ) :
  Message { unobservable. }.

message_items : StateItems.

choose
{
  //send message
  {unobservable.
  via sendMsg receive message_in.
  message_items := analyzeMessage(message_in).
  if localCommunication(message_items, addresses)
  then //deliver message locally
    via receiveMsg send message_in.
    behavior().
  else //deliver message remotely
    transmission_out :=
      generateTransmission(message_state, addresses).
    via transmission send transmission_out.
    behavior().}

  or

  //deliver message
  {unobservable;
  via deliver receive transmission_in.
  transmission_data :=
    analyzeTransmission(transmission_in, addresses).
  message_out := generateMessage(transmission_data, addresses).
  via receiveMsg send message_out.
  behavior().}
}

```

A.5 Synchronization

```

component Synchronization is abstraction ()
{
  type SynchronizationUpdate is Any.
  type StateItem is view[name : String, val : Any].
  type StateItems is set[StateItem].
  type SynchronizationItems is location[set[StateItems]].

  port synchronize is { connection syncOut is out(SynchronizationUpdate).
                       connection syncIn is in(SynchronizationUpdate) }.
  port readWriteState is {
    connection stateTemplate is out(StateItems).
    connection readState is in(StateItems).

  }

  synchronizationItems : SynchronizationItems.

  behavior is {
    selectItems is function( items : SynchronizationItems ) :
      StateItems { unobservable. }.
    localSync is function( state : StateItems,
      items : SynchronizationItems :
      Boolean { unobservable. }.
  }
}

```

```

updateState is function( state: StateItems ) :
  StateItems { unobservable. }.
generateSyncUpdate is function( state : StateItems ) :
  SynchronizationUpdate { unobservable. }.
decodeSyncUpdate is function( update : SynchronizationUpdate ) :
  StateItems { unobservable. }.

choose
{
  //locally initiated synchronization
  {unobservable.
  via stateTemplate send selectItems(synchronizationItems).
  via readState receive state_items : StateItems.
  if localSync(state_items, synchronizationItems)
  then {
    //local state update
    via writeState send updateState(state_items).
  }
  else {
    //external synchronization
    unobservable.
    via syncOut send generateSyncUpdate(state_items).
  }
  behavior().}
or
  //remotely initiated synchronization
  {unobservable.
  via syncIn receive sync_update : SynchronizationUpdate.
  via writeState send decodeSyncUpdate(sync_update).
  behavior().}
}
}

```

B. SITUATED AGENT

```

component SituatedAgent is abstraction ()
{
  //type definitions
  type Focus is view[focusName : String, focusParams : Any].
  type Sense is view[agentId : String, focus : Focus].
  type Representation is Any.
  type Action is view[agentId : String, actionName : String,
    actionParams : Any].
  type Message is view[id : Integer, sender : String, receiver :
    protocol : String, performative : String, content : Any].
  type KnowledgeItem is view[name : String, val : Any].
  type Knowledge is set[KnowledgeItem].

  port perceive is { connection sense is out(Sense).
    connection sensed is in(Representation) }.
  port act is { connection action is out(Action) }.
  port communicate is { connection sendMsg is out(Message).
    connection receiveMsg is in(Message) }.

  //composition

```

```

behavior is { compose {
    perception is Perception().
    and communication is Communication().
    and actionSelection is ActionSelection().
    and currentKnowledge is CurrentKnowledge().
} } where {
    perception::sense relays sense
    and perception::request unifies communication::request
    and perception::request unifies actionSelection::request
    and sensed relays perception::sensed
    and communication::sendMsg relays sendMsg
    and receiveMsg relays communication::receiveMsg
    and actionSelection::action relays action
    and currentKnowledge::readWriteKnowledge unifies
        communication::readWriteKnowledge
    and currentKnowledge::readWriteKnowledge unifies
        actionSelection::readWriteKnowledge
    and currentKnowledge::readWriteKnowledge unifies
        perception::readWriteKnowledge
}
}

```

B.1 Current Knowledge

```

component CurrentKnowledge is abstraction()
{
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItem].
    type KnowledgeRepository is location[set[Knowledge]].

    port readWriteKnowledge is {
        connection knowledgeTemplate is in(Knowledge).
        connection readKnowledge is out(Knowledge).
        connection writeKnowledge is in(Knowledge) }.
        assuming { protocol is { (via knowledgeTemplate receive Knowledge.
            true*.via readState send Knowledge.)*
            or (via writeState receive Knowledge.)* }}.

    currentKnowledge : KnowledgeRepository.

    behavior is {
        write is function( knowledge : Knowledge,
            repository : KnowledgeRepository ) :
            KnowledgeRepository { unobservable. }.
        read is function( template : Knowledge,
            repository : KnowledgeRepository ) :
            Knowledge { unobservable. }.

        choose
        {
            //read knowledge
            {via knowledgeTemplate receive knowledge_template : Knowledge.
            unobservable.
            via readKnowledge send read(knowledge_template, currentKnowledge).
            behavior().}
        or

```

```

        //write knowledge
        {via writeKnowledge receive knowledge_items : Knowledge.
        unobservable.
        currentKnowledge := write(knowledge_items, currentKnowledge).
        behavior().}
    }
}

```

C. SELECTIVE PERCEPTION

```

component Perception is abstraction()
{
    type Focus is view[focusName : String, focusParams : Any].
    type Filter is view[filterName : String, filterParams : Any].
    type PerceptionRequest is view[id : String, focus : Focus, filter : Filter].
    type Sense is view[agentId : String, focus : Focus].
    type Representation is Any.
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItem].

    port perceive is { connection sense is out(Sense).
                      connection sensed is in(Representation) }.
    port request is { connection requestPerception is in(PerceptionRequest)
                     connection notifyRequest is out(String) }.
    port readWriteKnowledge is {
        connection knowledgeTemplate is out(Knowledge).
        connection readKnowledge is in(Knowledge).
        connection writeKnowledge is out(Knowledge) }.

    behavior is { compose {
        sensing is Sensing().
        and interpreting is Interpreting().
        and descriptions is Descriptions().
        and filtering is Filtering().
    } } where {
        requestPerception relays sensing::applySensing
        and sensing::sense relays sense
        and sensed relays sensing::sensed
        and sensing::interpret unifies interpreting::interpret
        and interpreting::readDescriptions unifies
            descriptions::readDescriptions
        and interpreting::filter unifies filtering::filter
        and requestPerception relays filtering::requestPerception
        and filtering::readWriteKnowledge relays readWriteKnowledge
        and filtering::notifyRequest relays notifyRequest
    }
}

```

C.1 Sensing

```

component Sensing is abstraction()
{
    type Focus is view[focusName : String, focusParams : Any].
    type Filter is view[filterName : String, filterParams : Any].
    type PerceptionRequest is view[id : String, focus : Focus, filter : Filter].

```

```

type Sense is view[agentId : String, focus : Focus].
type Representation is Any.

port perceive is { connection sense is out(Sense).
                  connection sensed is in(Representation) }.
port apply is { connection applySensing is in(PerceptionRequest) }.
port interpret is { connection percept is out(Representation) }.

behavior is {
  generateSense is function(request : PerceptionRequest) :
    Sense { unobservable. }.

  via applySensing receive request : PerceptionRequest.
  via sense send generateSense(request).
  unobservable.
  via sensed receive representation : Representation.
  unobservable.
  via percept send representation.
  behavior().
}

```

C.2 Descriptions

```

component Descriptions is abstraction()
{
  type Representation is Any.
  type Description is Any.
  type DescriptionRepository is location[set[Description]].

  port readDescriptions is {
    connection representation is in(Representation).
    connection description is out(Description) }
    assuming { protocol is { (via representation receive Any.true*.
      via description send Any.)* } }.

  descriptions : DescriptionRepository.

  behavior is {
    select is function(rep : Representation,
      des : DescriptionsRepository) :
      Description { unobservable. }.

    unobservable.
    via representation receive repres : Representation.
    via description send select(repres, descriptions).
    behavior().
  }
}

```

C.3 Interpreting

```

component Interpreting is abstraction()
{
  type Representation is Any.
  type Description is Any.
  type KnowledgeItem is view[name : String, val : Any].

```

```

type Knowledge is set[KnowledgeItems].

port interpret is { connection interpretIn is in(Representation) }.
port readDescriptions is {
    connection representation is out(Representation).
    connection description is in(Description) }.
port filter is { connection interpretOut is out(Knowledge) }.

behavior is {
    interpreting is function(r : Representation, ds : Description) :
        Knowledge { unobservable. }.

    via interpretIn receive repres : Representation.
    via representation send repres.
    unobservable.
    via description receive descript : Description.
    via interpretOut send interpreting(repres, descript).
    behavior().
}
}

```

C.4 Filtering

```

component Filtering is abstraction()
{
    type Filter is view[filterName : String, filterParams : Any].
    type PerceptionRequest is view[id : String, focus : Focus, filter : Filter].
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItems].

    port filter is { connection filterIn is in(Knowledge) }.
    port apply is { connection applyFiltering is in(PerceptionRequest).
        connection notifyRequest is out(String) }.
    port readWriteKnowledge is {
        connection knowledgeTemplate is out(Knowledge).
        connection readKnowledge is in(Knowledge).
        connection writeKnowledge is out(Knowledge) }.

    behavior is {
        filtering is function(percept : Knowledge, filter : Filter) :
            Knowledge { unobservable. }.

        unobservable.
        via applyFiltering receive request : PerceptionRequest.
        unobservable.
        via filterIn receive percept : Knowledge.
        via writeKnowledge send filtering(percept, request::filter).
        via notifyRequest send request::id.
        behavior().
    }
}

```

D. BEHAVIOR-BASED ACTION SELECTION

```

component ActionSelection is abstraction()
{
    type Filter is view[filterName : String, filterParams : Any].

```

```

type PerceptionRequest is view[id : String, focus : Focus, filter : Filter].
type KnowledgeItem is view[name : String, val : Any].
type Knowledge is set[KnowledgeItems].
type Action is view[agentId : String, actionName : String,
    actionParams : Any].

port act is { connection invoke is out(Action) }.
port request is {
    connection requestPerception is out(PerceptionRequest).
    connection notifyRequest is in(String) }.
port readWriteKnowledge is {
    connection knowledgeTemplate is out(Knowledge).
    connection readKnowledge is in(Knowledge).
    connection writeKnowledge is out(Knowledge) }.

behavior is { compose {
    decisionMaking is DecisionMaking().
    and roles is Roles().
    and commitments is Commitments().
} } where {
    decisionMaking::updateReadRoles unifies roles::updateReadRoles
    and decisionMaking::updateReadCommitments unifies
        commitments::updateReadCommitments
    and decisionMaking::act relays act
    and decisionMaking::knowledgeTemplate relays knowledgeTemplate
    and readKnowledge relays decisionMaking::readKnowledge
    and decisionMaking::writeKnowledge relays writeKnowledge
    and decisionMaking::requestPerception relays requestPerception
    and notifyRequest relays decisionMaking::notifyRequest
}
}

```

D.1 Roles

```

component Roles is abstraction()
{
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItems].
    type Role is view[name : String, behaviors : Any,
        actions : set[Action]].

    port updateReadRoles is {
        connection updateRoles is in(Knowledge).
        connection readRoles is out(set[Role]).
    }

    roles : set[Role].

    behavior is {
        update is function(k : Knowledge, roles : set[Role]) :
            set[Role] { unobservable. }.

        choose
        {
            //update roles
            {via updateRoles receive knowledge : Knowledge.

```

```

    unobservable.
    roles := update(knowledge, roles).
    behavior().}
  or
    //read roles
    {unobservable.
    via readRoles send(roles).
    behavior().}
}
}

```

D.2 Commitments

```

component Commitments is abstraction()
{
  type KnowledgeItem is view[name : String, val : Any].
  type Knowledge is set[KnowledgeItems].
  type SituatedCommitment is view[name : String, context : Any,
    condition : Boolean, effect : Any].

  port updateReadCommitments is {
    connection updateCommitments is in(Knowledge).
    connection readCommitments is out( set[SituatedCommitment] ).
  }

  commitments : set[SituatedCommitment].

  behavior is {
    update is function(k : Knowledge, cs : set[SituatedCommitment]) :
      set[SituatedCommitment] { unobservable. }.

    choose
    {
      //update commitments
      {via updateCommitments receive knowledge : Knowledge.
      unobservable.
      roles := update(knowledge, commitments).
      behavior().}
    or
      //read commitments
      {unobservable.
      via readCommitments send(commitments).
      behavior().}
    }
  }
}

```

D.3 Decision Making

```

component DecisionMaking is abstraction()
{
  type KnowledgeItem is view[name : String, val : Any].
  type Knowledge is set[KnowledgeItems].
  type Role is view[name : String, behaviors : Any,
    actions : set[Action]].
  type SituatedCommitment is view[name : String, context : Any,
    condition Boolean, effect : Any].

```

```

type Action is view[agentId : String, actionName : String,
    actionParams : Any].
type ExternalActions is location[set[Action]].

port updateReadRoles is {
    connection updateRoles is in(Knowledge).
    connection readRoles is out(set[Role]). }
port updateReadCommitments is {
    connection updateCommitments is in(Knowledge).
    connection readCommitments is out(set[SituatedCommitment]). }
port act is { connection selectAction is out(Action). }.
port request is {
    connection requestPerception is out(PerceptionRequest).
    connection notifyRequest is in(String) }.
port readWriteKnowledge is {
    connection knowledgeTemplate is out(Knowledge).
    connection readKnowledge is in(Knowledge).
    connection writeKnowledge is out(Knowledge) }.

externalActions : ExternalActions.

behavior is {
    selectAction is function (roles : set[Role],
        commitments : set[SituatedCommitment] :
            Action { unobservable. }.
    externalAction is function (action : Action,
        actions : ExternalActions) :
            Boolean { unobservable. }.

    selected_action : Action.

    unobservable.
    via updateRoles send role_knowledge : Knowledge.
    via updateCommitments send commitment_knowledge : Knowledge.
    via readRoles receive roles : set[Role].
    via readCommitments receive commitments : set[SituatedCommitment].
    selected_action := selectAction(roles, commitments).
    if (externalAction(selected_action, externalActions))
    then {
        via selectAction send selected_action.
        behavior().
    }
    else {
        unobservable.
        behavior().
    }
}
}

```

E. PROTOCOL-BASED COMMUNICATION

```

component Communication is abstraction()
{
    type Filter is view[filterName : String, filterParams : Any].
    type PerceptionRequest is view[id : String, focus : Focus, filter : Filter].
    type KnowledgeItem is view[name : String, val : Any].

```

```

type Knowledge is set[KnowledgeItems].
type Message is view[id : Integer, sender : String, receiver : String,
  cprotocol : String, performative : String, content : Any].

port communicate is {
  connection sendMsg is out(Message).
  connection receiveMsg is in(Message) }.
port request is {
  connection requestPerception is out(PerceptionRequest).
  connection notifyRequest is in(String) }.
port readWriteKnowledge is {
  connection knowledgeTemplate is out(Knowledge).
  connection readKnowledge is in(Knowledge).
  connection writeKnowledge is out(Knowledge) }.

behavior is { compose {
  messageReceiving is MessageReceiving().
  and inbox is Inbox().
  and messageDecoding is MessageDecoding().
  and communicating is Communicating().
  and conversations is Conversations().
  and messageEncoding is MessageEncoding().
  and outbox is Outbox().
  and messageSending is MessageSending().
} } where {
  receiveMsg relays messageReceiving::receiveMsg
  and messageReceiving::addIn unifies inbox::addIn
  and messageDecoding::removeIn unifies inbox::removeIn
  and messageDecoding::decode unifies communicating::decode
  and communicating::knowledgeTemplate relays knowledgeTemplate
  and readKnowledge relays communicating::readKnowledge
  and communicating::writeKnowledge relays writeKnowledge
  and communicating::requestPerception relays requestPerception
  and notifyRequest relays communicating::notifyRequest
  and communicating::readUpdateConversation unifies
    conversations::readUpdateConversation
  and communicating::encode unifies messageEncoding::encode
  and messageEncoding::addOut unifies outbox::addOut
  and messageSending::removeOut unifies outbox::removeOut
  and messageSending::sendMsg relays sendMsg
}
}

```

E.1 Message Receiving

```

component MessageReceiving is abstraction ()
{
  type Message is view[id : Integer, sender : String, receiver : String,
    cprotocol : String, performative : String, content : Any].

  port communicate is { connection receiveMsg is in(Message) }.
  port addInbox is { connection addIn is out(Message) }.

  behavior is {
    unobservable.
    via receiveMsg receive message : Message.
  }
}

```

```

        via addIn send message.
        behavior().
    }
}

```

E.2 Inbox

```

component Inbox is abstraction()
{
    type Message is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : Any].
    type MessageRepository is location[set[Message]].

    port addInbox is { connection addIn is in(Message) }.
    port removeInbox is { connection removeIn is out(Message) }.

    messageRepository : MessageRepository.

    behavior is {
        addMsg is function( m : Message, rep : MessageRepository ) :
            MessageRepository { unobservable. }.
        removeMsg is function( rep : MessageRepository ) :
            Message { unobservable. }.

        unobservable.
        choose {
            //add message
            {via addIn receive message_in : Message.
                messageRepository := addMsg(message_in, messageRepository).
                behavior().}
            or
            //remove message
            {if (# messageRepository > 0)
                then {
                    via removeIn send removeMsg(messageRepository).
                    behavior().}
                else {
                    //empty inbox
                    behavior().}}
        }
    }
}

```

E.3 Message Decoding

```

component MessageDecoding is abstraction()
{
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItem].
    type Message is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : Any].
    type ContentData is sequence[Knowledge].
    type MessageData is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : ContentData].

    port decodeMessage is { connection decode is out(MessageData) }.
    port removeInbox is { connection removeIn is in(Message) }.
}

```

```

language : Any.
ontology : Any.

behavior is {
  decode is function( msg : Message, ontol : Any, lang : Any ) :
    MessageData { unobservable. }.

  unobservable.
  via removeIn receive message : Message.
  via decodeMsg send decode(message, language, ontology).
  behavior().
}
}

```

E.4 Conversations

```

component Conversations is abstraction()
{
  type KnowledgeItem is view[name : String, val : Any].
  type Knowledge is set[KnowledgeItem].
  type ContentData is sequence[Knowledge].
  type MessageData is view[id : Integer, sender : String, receiver : String,
    cprotocol : String, performative : String, content : ContentData].
  type Conversation is sequence[MessageData].
  type ConversationRepository is location[set[Conversation]].

  port readUpdateConversation is {
    connection selectConversation is in(Integer).
    connection readConversation is out(Conversation).
    connection updateConversation is in(MessageData).
    connection addConversation is in(MessageData).
    connection terminateConversation is in(Integer) }.

  conversations : ConversationRepository.

  behavior is {
    add is function(data : MessageData,
      convs : ConversationRepository ) :
      ConversationRepository { unobservable. }.
    read is function( id : Integer,
      convs : ConversationRepository ) :
      Conversation { unobservable. }.
    write is function( data : MessageData,
      convs : ConversationRepository ) :
      ConversationRepository { unobservable. }.
    terminate is function(id : Integer,
      ConversationRepository) :
      ConversationRepository { unobservable. }.

    choose {
      //add conversation
      {via addConversation receive data : MessageData.
        conversations := add(data, conversations).
        behavior().}
    or

```

```

        //read conversation
        {via selectConversation receive id : Integer.
        via readConversation send read(id, conversations).
        behavior().}
    or
        //write conversation
        {via updateConversation receive data : MessageData.
        conversations := write(data, conversations).
        behavior().}
    or
        //delete conversation
        {via terminateConversation receive id : Integer.
        conversations := terminate(id, conversations).
        behavior().}
    }
}
}

```

E.5 Communicating

```

component Communicating is abstraction ()
{
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItem].
    type ContentData is sequence[Knowledge].
    type MessageData is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : ContentData].
    type Conversation is sequence[MessageData].
    type ConversationIDs is set[Integer].
    type CProtocol is view[name : String, steps : Any].
    type Protocols is set[CProtocol].

    port readUpdateConversation is {
        connection selectConversation is in(Integer).
        connection readConversation is out(Conversation).
        connection updateConversation is in(MessageData).
        connection addConversation is in(MessageData).
        connection terminateConversation is in(Integer) }.
    port decodeMessage is { connection decode is in(MessageData) }.
    port encodeMessage is { connection encode is out(MessageData) }.
    port request is {
        connection requestPerception is out(PerceptionRequest).
        connection notifyRequest is in(String) }.
    port readWriteKnowledge is {
        connection knowledgeTemplate is out(Knowledge).
        connection readKnowledge is in(Knowledge).
        connection writeKnowledge is out(Knowledge) }.

    protocols : Protocols.
    conversationIds : ConversationIDs.

    behavior is {
        extractKnowledge is function (data : MessageData) :
            Knowledge { unobservable. }.
        addCommunication is function (data : MessageData) :
            Knowledge { unobservable. }.
    }
}

```

```

updateCommunication is function (data : MessageData) :
    Knowledge { unobservable. }.
closeCommunication is function (c : Conversation) :
    Knowledge { unobservable. }.
ongoingConversation is function (id : Integer, ids : ConversationIDs) :
    Boolean { unobservable. }.
continueConversation is function (k : Knowledge, ids : ConversationIDs) :
    Boolean { unobservable. }.
terminateConversation is function (ps : Protocols,
    c: Conversation, k : Knowledge) : Boolean { unobservable. }.
initiateConversation is function (ps : Protocols, k : Knowledge) :
    MessageData { unobservable. }.
encodeMessage is function (ps : Protocols, c : Conversation,
    k : Knowledge) : MessageData { unobservable. }.
getId is function (k : Knowledge, ids :ConversationIDs) :
    Integer { unobservable. }.
addId is function (id : Integer, ids :ConversationIDs) :
    ConversationIDs { unobservable. }.

msg_knowledge : Knowledge.
enc_msg_data : Messagedata.
conv_id : Integer.

choose
{
    //handle incoming message
    via decode receive dec_msg_data : MessageData.
    msg_knowledge := extractKnowledge(dec_msg_data).
    if(ongoingConversation(dec_msg_data::id, conversationIds))
    then {
        via updateConversation send dec_msg_data.
        via writeKnowledge send updateCommuncation(dec_msg_data).
    }
    else {
        via addConversation send dec_msg_data.
        via writeKnowledge send addCommuncation(dec_msg_data).
    }
    behavior().
}
or
{
    //handle outgoing message
    unobservable.
    via readKnowledge receive knowledge : Knowledge.
    if(continueConversation(knowledge, conversationIds))
    then {
        via selectConversation send getId(knowledge, conversationIds).
        via readConversation receive conversation : Conversation.
        enc_msg_data :=
            encodeMessage(protocols, conversation, knowledge).
        via encode send enc_msg_data.
        via updateConversation send enc_msg_data.
        via writeKnowledge send updateCommunication(enc_msg_data).
    }
    else {
        enc_msg_data := initiateConversation(protocols, knowledge).
    }
}

```

```

        via addConversation send enc_msg_data.
        conversationIds := addId(id, conversationIds).
        via writeKnowledge send addCommunication(enc_msg_data).
    }
    behavior().
}
or
{
    unobservable.
    via readKnowledge receive knowledge : Knowledge.
    conv_id := getId(knowledge, conversationIds).
    via selectConversation send conv_id.
    via readConversation receive conversation : Conversation.
    if(terminateConversation(protocols, conversation, knowledge))
    then {
        //terminate conversation
        via terminateConversation send conv_id.
        conversationIds := deleteId(conv_id, conversationIds).
        via writeKnowledge send closeCommunication(conversation).
    }
    else {
        //silence
        unobservable.
    }
    behavior().
}
}
}

```

E.6 Message Encoding

```

component EncodingDecoding is abstraction ()
{
    type KnowledgeItem is view[name : String, val : Any].
    type Knowledge is set[KnowledgeItems].
    type Message is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : Any].
    type ContentData is sequence[Knowledge].
    type MessageData is view[id : Integer, sender : String, receiver : String,
        cprotocol : String, performative : String, content : ContentData].

    port encodeMessage is { connection encode is in(MessageData) }.
    port addOutbox is { connection addOut is out(Message) }.

    language : Any.
    ontology : Any.

    behavior is {
        encode is function( data : MessageData, ontol : Any, lang : Any ) :
            Message { unobservable. }.

        unobservable.
        via encode receive message_data : MessageData.
        via addOut send encode(message_data, language, ontology).
        behavior().
    }
}

```

```
}

```

E.7 Outbox

```
component Outbox is abstraction()
{
  type Message is view[id : Integer, sender : String, receiver : String,
    cprotocol : String, performative : String, content : Any].
  type MessageRepository is location[set[Message]].

  port addOutbox is { connection addOut is in(Message) }.
  port removeOutbox is { connection removeOut is out(Message) }.

  messageRepository : MessageRepository.

  behavior is {
    addMsg is function( m : Message, rep : MessageRepository ) :
      MessageRepository { unobservable. }.
    removeMsg is function( rep : MessageRepository ) :
      Message { unobservable. }.
    unobservable.
    choose {
      //add message
      {via addIn receive message_in : Message.
        messageRepository := addMsg(message_in, messageRepository).
        behavior(). }
    or
      //remove message
      {if (# messageRepository > 0)
        then {
          via removeIn send removeMsg(messageRepository).
          behavior().}
        else {
          //empty outbox
          behavior(). }
        }
    }
  }
}
```

E.8 Message Sending

```
component MessageSending is abstraction ()
{
  type Message is view[id : Integer, sender : String, receiver : String,
    cprotocol : String, performative : String, content : Any].

  port removeOutbox is { connection removeOut is out(Message) }.
  port communicate is { connection sendMsg is in(Message) }.

  behavior is {
    unobservable.
    via removeOut receive message.
    via sendMsg send message : Message.
    behavior().
  }
}
```

THIS DOCUMENT IS THE ONLINE-ONLY APPENDIX TO:

An Architectural Style for Self-Adaptive Multi-Agent Systems

DANNY WEYNS

KU Leuven, Belgium and Linnaeus University, Sweden

FLAVIO OQUENDO

IRISA - Université de Bretagne-Sud, France

ACM Transactions Journal Format, Vol. 2, No. 3, 09 2001, Pages .

In this appendix we list the patterns that were studied in the systematic literature review.

Patterns	Reference
Itinerary, Master-Slave, Meeting, Plan, Ticket, Facilitator, Forwarding, Organized group, Locker, Messenger	[Aridor and Lange 1998]
The Layered Agent, Reactive Agent, Deliberative Agent, Opportunistic Agent, Interface Agent, Intention, Prioritizer, Adaptable Active Object, Message Forwarder, Plan as Command, Plan and Intention Factory, Conversation, Facilitator, Agent Proxy, Protocol, Emergent Society, Clone, Remote Configurator, Broker, Migration Thread Factory, Agent Builder, Layer Linker	[Kendall et al. 1998]
Basic Mobility, Itinerary, Star-shaped Movement, Branching, Contract net protocol, Cooperation Protocol Pattern, Direct Interaction, Mediation, Dispatching	[Tahara et al. 1999]
Broker, Embassy, Mediator, Monitor, Wrapper	[Hayden et al. 1999]
Metamorphic Architecture Pattern, Mediator, Task Decomposition Pattern, Virtual Clustering Pattern, The Partial Cloning Pattern, The Prototyping Pattern	[Shu and Norrie 1999]
Direct Coupling Pattern, Proxy Agent Pattern, Communication Sessions Pattern, Badges, Event Dispatcher	[Deugo et al. 1999]
Sentinel Agent Behavior Pattern	[Chacon et al. 2000]
Receptionist, Session Pattern, Secretary, Mobile Session, Antenna, Private Session, Meeting with Moderator	[Silva et al. 2000]
Blackboard, Market Maker, Meeting, Master-Slave, Negotiating Agents	[Deugo et al. 2001]
Synchronizer, Environment Mediated Communication, Updating shared state, Behavior-based Decision	[Schelfhout et al. 2002]
InteRRaP, Contract net protocol	[Lind 2002]

Permission to make digital/hard copy of all or part of this material without fee for personal or classroom use provided that the copies are not made or distributed for profit or commercial advantage, the ACM copyright/server notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists requires prior specific permission and/or a fee.

© 2012 ACM 0000-0000/2001/0000-0001 \$5.00

Agency Guard, Agent Authenticator, Sandbox, Access Controller	[Mouratidis et al. 2003]
Role Agent Pattern	[Cabri et al. 2003]
Agency Guard, Agent Authenticator, Sandbox, Access Controller	[Mouratidis et al. 2003]
Role Agent Pattern	[Cabri et al. 2003]
Monitor, Broker, Matchmaker, Mediator, Embassy, Booking, Call-For-Proposal, Bidding	[Kolp et al. 2003]
Reflective Blackboard	[Silva et al. 2003]
Organisation schemes, Protocols, Marks, Influences, BDI architecture, Vertical architecture, Horizontal architecture, Recursive architecture, Iniquity, Discretisation, Physical entity	[Sauvage 2004]
Hybrid Recogniser, Sense-and-Infer, Assisted Sense and Infer, Ecological Recogniser, Assisted Ecological Recogniser, Clairvoyant	[Heinze 2004]
Market-organiser, Agent-side comparison-shopping, Server-side comparison-shopping pattern, Itinerary-balancing pattern	[Nguyen and Yang 2004]
Pipeline	[Gonzalez-Palacios and Luck 2004]
Agent Society, Agent as Delegate, Agent as Mediator, Common Vocabulary, User Agent, Task Agent, Resource Agent	[Weiss 2004]
Multi-agent layer pattern, Web usage mining, WUMA-Interface, WUMA-UserM, WUMA-Acquirer, WUMA-Miner	[Girardi et al. 2005]
Rule Composer, Decision Tree	[Taylor et al. 2005]
Service Client Pattern, Service Representative	[Muller et al. 2005]
Initiator, Observer	[Mohamad et al. 2006]
Structure-in-5, Pyramid, Chain of values, The matrix, Bidding, Joint venture, Arm's-length, Hierarchical contracting, Co-optation	[Kolp et al. 2006]
Actors, Refine Actor Goals, Means-End Analysis, Contribution Analysis, AND/OR Decomposition, Ask for Help, Refine Models, System-to-Be	[Mouratidis and Weiss 2006]
Replication, Collective Sort, Evaporation, Aggregation, Diffusion	[Gardelli et al. 2007]
Behaviour Helper Pattern	[Charan Ojha et al. 2007]
Gradient Fields, Market-Based Control	[De Wolf et al. 2007]
Agent Interface, SCADA control sequence agent, Agent optimization, Agent NET-MAP pattern	[Eichelkraut et al. 2007]
Virtual Environment, Situated Agent, Selective Perception, Roles & Situated Commitments, Protocol-based Communication	[Weyns 2009]
Perception Memory Pattern, Exponential Growth Pattern	[Kluegl et al. 2009]
Disciplined flood, Propertinerary, Smart message, Delegate ant MAS, Delegate MAS	[Holvoet et al. 2010]
Planner, VFHPlanner Pattern, A*Planner, Persistence agent, Memento agent, Social agent, Explorer, Sequential-resource share, Parallel resource share, Query, Inform, Request, Secure Query, Secure Request, Secure Inform, Contract net protocol, Publish-subscribe, Information agent, Holonic society, Supply chain	[Chella et al. 2010]
Selection of Relevant Source Material	[Cheah et al. 2010]
Policy-based	[Guo et al. 2011]
Scheduler Scramble, Context and Projection Hierarchy, Diffuser, Strategy, Logo World, Learning, Model-View-Controller, Double Buffer	[North and Macal 2011]
Aggregation, Spreading, Gossip	[Fernandez-Marquez et al. 2011]
Composite DelegateMAS	[Cruz Torres et al. 2011]